Virtual Worlds

Programming Assignment #3 Report



학과/학부	소프트웨어학부
학번	2021203015
이름	강선후

2025. 6. 15

■ Fulfill table

Easy 1	Camera switching	O
Easy 2	Grid map	O
Medium 1	Setting up objects	O
Medium 2	Changing the color of the grid cells	X
Hard 1	Move the player along a Bezier curve	O
Hard 2	Path Finding using BFS	O
Hard 3	Path Finding using Greedy	O
Expert 1	Path Finding using A*	O
Expert 2	Comparing the algorithms	X

■ For each requirement, explain how you fulfilled it

☐ Easy 1: Camera switching

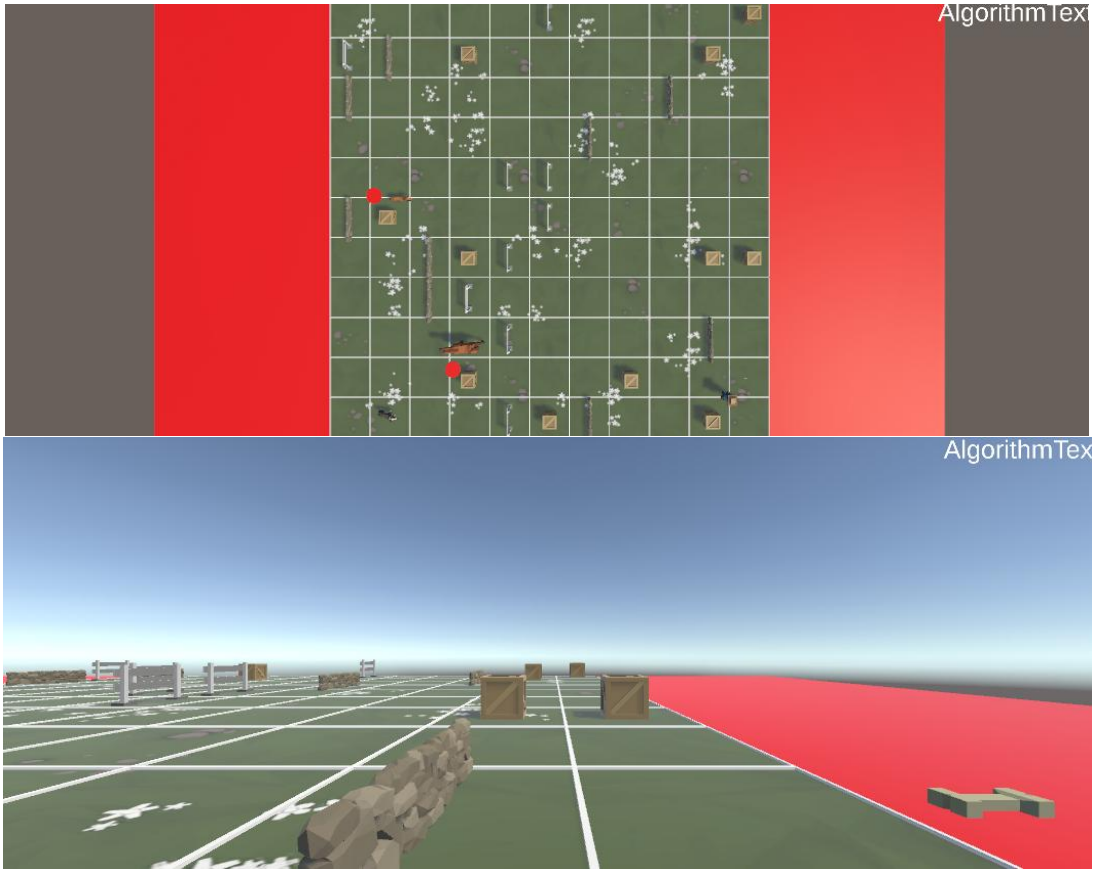
```
void Update()
{
    if (Input.GetKeyDown(KeyCode.LeftShift))
    {
        isFirstPerson = !isFirstPerson;

        if (isFirstPerson)
            ActivateFirstPerson();
        else
            ActivateThirdPerson();
    }
}
```

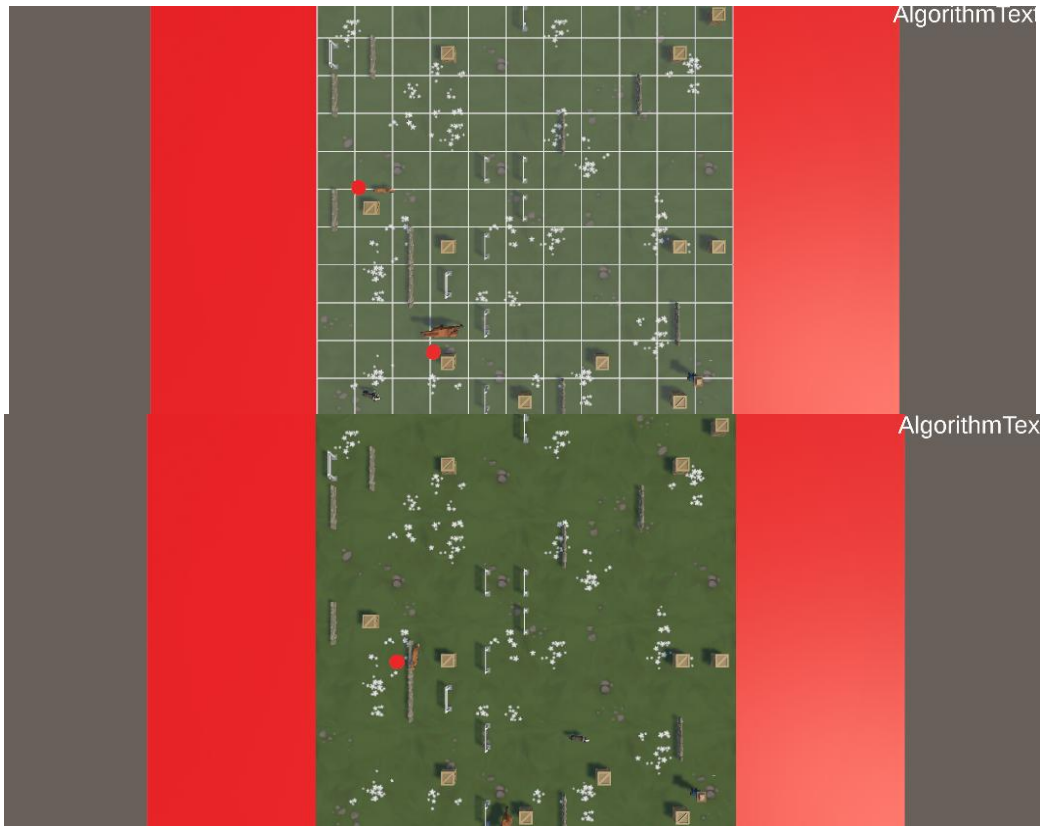
```
void ActivateFirstPerson()
{
    firstPersonCam.enabled = true;
    thirdPersonCam.enabled = false;
}

참조 1개
void ActivateThirdPerson()
{
    firstPersonCam.enabled = false;
    thirdPersonCam.enabled = true;
}
```

Shift 키 입력을 통해 1인칭과 3인칭 카메라를 전환할 수 있도록 하였습니다. 3인칭 카메라는 맵 전체를 위에서 조망할 수 있도록 배치하였으며, 각 모드 전환 시 카메라 활성화 상태가 전환되도록 구현했습니다.



☐ Easy 2: Grid map



GridManager를 통해 11x11의 격자를 맵 위에 자동으로 배치하였고, G 키를 누르면 격자 시각화를 토글할 수 있도록 했습니다. 격자를 벗어난 영역에는 플레이어나 동물이 이동할 수 없도록 이동 제한도 적용했습니다.

```
GridCell cellData = cell.GetComponent<GridCell>();
if (cellData != null)
{
    cellData.x = x;
    cellData.z = z;
    cellData.isBlocked = !IsInsideBounds(x, z);
}
```

```
void ToggleGridVisibility()
{
    gridVisible = !gridVisible;

    foreach (GameObject cell in gridCells)
    {
        if (cell != null)
            cell.SetActive(gridVisible);
    }
```

□ Medium 1: Setting up objects

시작 시 각 동물(말, 개, 여우)을 각각 1마리씩 무작위 위치에 배치하며, 각 셀마다 일정 확률로 장애물이 생성되도록 구현했습니다. 확률은 public 변수로 선언되어 Inspector에서 직접 조절할 수 있도록 했습니다. R 키를 누르면 전체 맵이 초기화되어 다시 배치되도록 하였습니다.

```
foreach (GridCell cell in allCells)
{
    if (Random.value < obstacleProbability)
    {
        GameObject prefab = obstaclePrefabs[Random.Range(0, obstaclePrefabs.Length)];
        GameObject obj = Instantiate(prefab, cell.transform.position, Quaternion.identity);
        spawnedObjects.Add(obj);
        cell.isBlocked = true;
    }
```

```
void SpawnAnimal(GameObject prefab, List<GridCell> cells)
{
    for (int i = 0; i < 100; i++)
    {
        GridCell cell = cells[Random.Range(0, cells.Count)];

        if (!cell.isBlocked && gridManager.IsInsideBounds(cell.x, cell.z))
        {
            GameObject obj = Instantiate(prefab, cell.transform.position, Quaternion.identity);
            spawnedObjects.Add(obj);
            cell.isBlocked = true;
            return;
        }
```



☐ Medium 2: Changing the color of the grid cells

여러 가지 방법을 시도했지만, 최종적으로 기능 구현에는 실패했습니다.
과제 의도는 제공된 머티리얼 하나를 기반으로 **동적으로 색상을 변경**하는 방식이라고 판단하여 SetColor()를 사용했으며, 실제 해당 함수 호출 시 Renderer에 색상이 변경되는 것처럼 **Editor 상 머티리얼 값이 실시간으로 바뀌는 것까지 확인**했습니다.

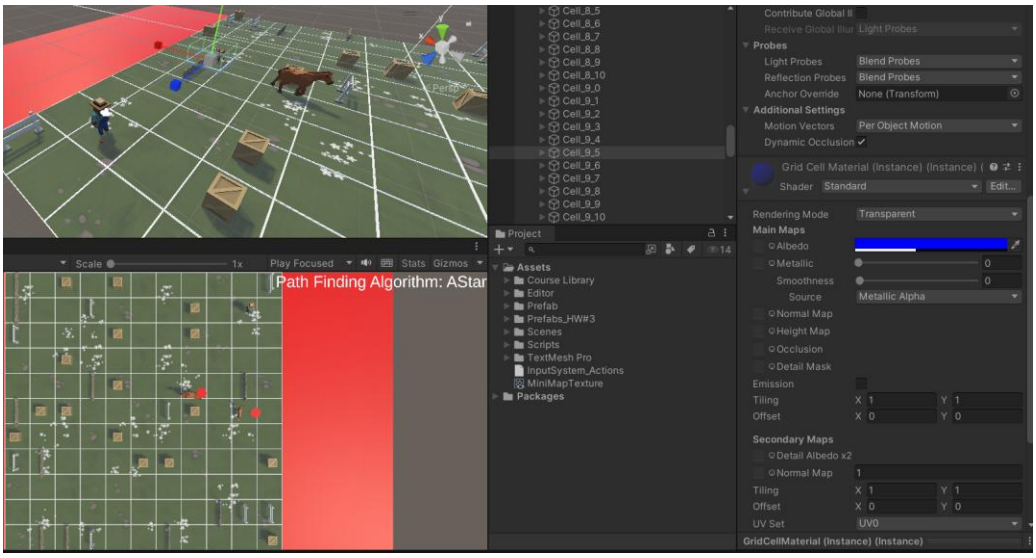
하지만 플레이 화면에서는 **변경된 색상이 전혀 보이지 않는 문제**가 발생했습니다. Shader 문제, 머티리얼 인스턴싱, 혹은 렌더러 캐싱 등의 원인을 의심하며 다양한 조치를 취했지만 결과는 동일했습니다.

차선책으로는 아예 **빨강/초록/파랑 머티리얼을 각각 따로 만들어** 해당 오브젝트의 머티리얼을 직접 교체하는 방식도 시도했으며, 로그 상에서는 정상적으로 교체가 이루어졌다고 출력되었지만 **결과적으로 화면상 변화는 없었습니다.**

마지막으로는 아예 Instantiate()를 사용해 해당 GridCell 프리팹을 새로 생성하여 교체하는 방법도 시도했으나 실패하게 되었고, **결국 해당 기능은 과제에서 제외**하게 되었습니다.

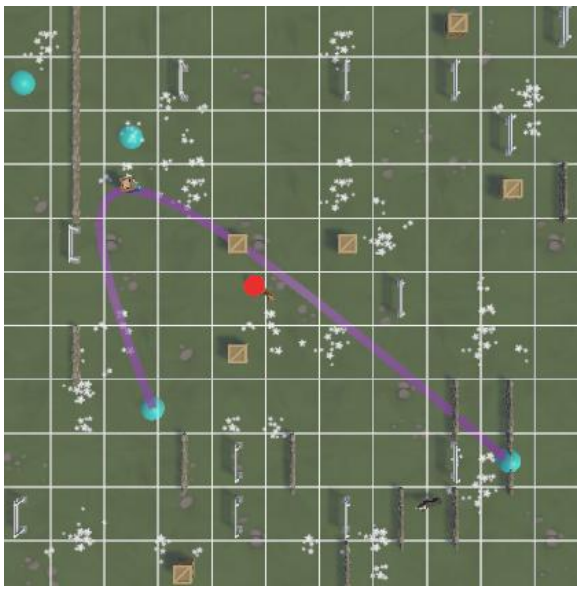
```
void HighlightNearest(Vector3 position, string colorName)
{
    GridCell nearest = FindNearestCell(position);
    if (nearest != null && !nearest.isBlocked)
    {
        Debug.Log($"하이라이트: {nearest.name} → {colorName}");
        nearest.SetColor(colorName);
    }
```

```
Console
Clear | Collapse | Error Pause | Editor
UnityEngine.Debug:Log (object)
[03:02:25] 하이라이트: Cell_7_7 → green
UnityEngine.Debug:Log (object)
[03:02:25] 하이라이트: Cell_0_6 → blue
UnityEngine.Debug:Log (object)
```



☐ Hard 1: Move the player along a Bezier curve

B 키를 누르면 현재 플레이어 위치를 시작점으로 하여 무작위 위치의 GridCell 3개를 골라 총 4개의 control point로 베지어 곡선을 생성하였습니다. 곡선을 따라 일정 시간에 걸쳐 부드럽게 이동하며, 중간에 키 입력이 들어올 경우 중단하거나 재시작할 수 있도록 했습니다. Control Point와 곡선은 prefab을 이용하여 시각화되며, 도달 후 자동 삭제됩니다. 곡선의 시각적 표현은 직접 Line Renderer 컴포넌트를 적용한 프리펩을 생성하여 이용하였습니다.





```
Vector3 CalculateBezier(float t)
{
    Vector3 p0 = controlPoints[0];
    Vector3 p1 = controlPoints[1];
    Vector3 p2 = controlPoints[2];
    Vector3 p3 = controlPoints[3];

    float u = 1 - t;
    float tt = t * t;
    float uu = u * u;
    float uuu = uu * u;
    float ttt = tt * t;

    Vector3 point = uuu * p0;
    point += 3 * uu * t * p1;
    point += 3 * u * tt * p2;
    point += ttt * p3;

    return point;
}
```

```
void StartBezierMovement()
{
    ClearVisuals();
    controlPoints.Clear();

    // 첫 포인트는 플레이어 현재 위치
    controlPoints.Add(transform.position);

    // 나머지 3개는 랜덤한 GridCell
    List<GridCell> cells = gridManager.GetAllCells();
    for (int i = 1; i < 4; i++)
    {
        int idx = Random.Range(0, cells.Count);
        controlPoints.Add(cells[idx].transform.position);
    }
```

☐ Hard 2: Path Finding using BFS

현재 들고 있는 음식에 따라 매칭되는 동물이 플레이어를 따라오도록 하였고, 그 외 동물은 자유롭게 이동하도록 구현했습니다. BFS 알고리즘을 사용해 경로를 탐색하고, 경로는 노란색으로 시각화될 수 있도록 코드를 작성하였고, 도중에 위치가 바뀌면 경로가 실시간으로 갱신되도록 하였습니다. Medium2에서의 이슈로 색이 바뀌는 것도 해당 오브젝트의 인스펙터 상으로는 확인하였으나 화면 출력은 되지 않았습니다.

```
List<GridCell> FindPathBFS(GridCell start, GridCell goal)
{
    Dictionary<GridCell, GridCell> cameFrom = new();
    Queue<GridCell> queue = new();
    queue.Enqueue(start);
    cameFrom[start] = null;

    while (queue.Count > 0)
    {
        GridCell current = queue.Dequeue();
        if (current == goal) break;

        foreach (var neighbor in GetNeighbors(current))
        {
            if (!cameFrom.ContainsKey(neighbor))
            {
                cameFrom[neighbor] = current;
                queue.Enqueue(neighbor);
            }
        }
    }

    return ReconstructPath(cameFrom, goal);
}
```



☐ Hard 3: Path Finding using Greedy

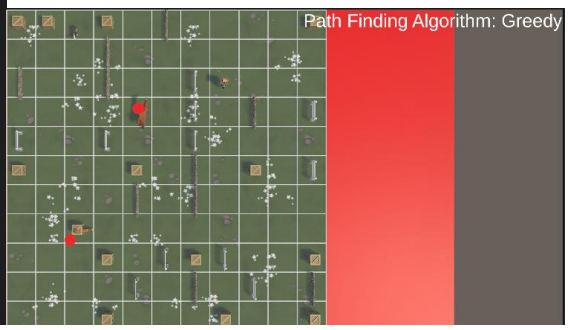
Greedy 알고리즘으로 경로 탐색을 수행하며, BFS와 같이 경로는 노란색으로 시각화 될 수 있도록 코드를 작성하였습니다. Priority Queue 클래스를 사용하여 휴리스틱 기반으로 가장 목표에 가까운 셀을 먼저 방문하도록 했습니다. 1, 2 키를 통해 BFS와 Greedy 알고리즘을 전환할 수 있으며, 현재 알고리즘은 우측 상단 텍스트로 표시됩니다.

```
List<GridCell> FindPathGreedy(GridCell start, GridCell goal)
{
    PriorityQueue<GridCell> frontier = new();
    Dictionary<GridCell, GridCell> cameFrom = new();
    frontier.Enqueue(start, 0);
    cameFrom[start] = null;

    while (frontier.Count > 0)
    {
        GridCell current = frontier.Dequeue();
        if (current == goal) break;

        foreach (var neighbor in GetNeighbors(current))
        {
            if (!cameFrom.ContainsKey(neighbor))
            {
                float priority = Vector2Int.Distance(
                    new Vector2Int(neighbor.x, neighbor.z),
                    new Vector2Int(goal.x, goal.z));
                frontier.Enqueue(neighbor, priority);
                cameFrom[neighbor] = current;
            }
        }
    }

    return ReconstructPath(cameFrom, goal);
}
```



☐ Expert 1: Path Finding using A*

A* 알고리즘을 통해 g(n)과 h(n)을 각각 Dijkstra 누적 거리와 Euclidean 거리로 계산하고, f(n) = g + h를 통해 우선순위를 산정해 경로를 탐색하도록 구현하였습니다. 3 키를 통해 A* 알고리즘을 선택할 수 있으며, 역시 경로는 노란색으로 표시되고자 하였고, 텍스트 UI를 통해 현재 알고리즘이 표시될 수 있도록 구현하였습니다.

```
List<GridCell> FindPathAStar(GridCell start, GridCell goal)
{
    PriorityQueue<GridCell> frontier = new();
    Dictionary<GridCell, GridCell> cameFrom = new();
    Dictionary<GridCell, float> costSoFar = new();

    frontier.Enqueue(start, 0);
    cameFrom[start] = null;
    costSoFar[start] = 0;

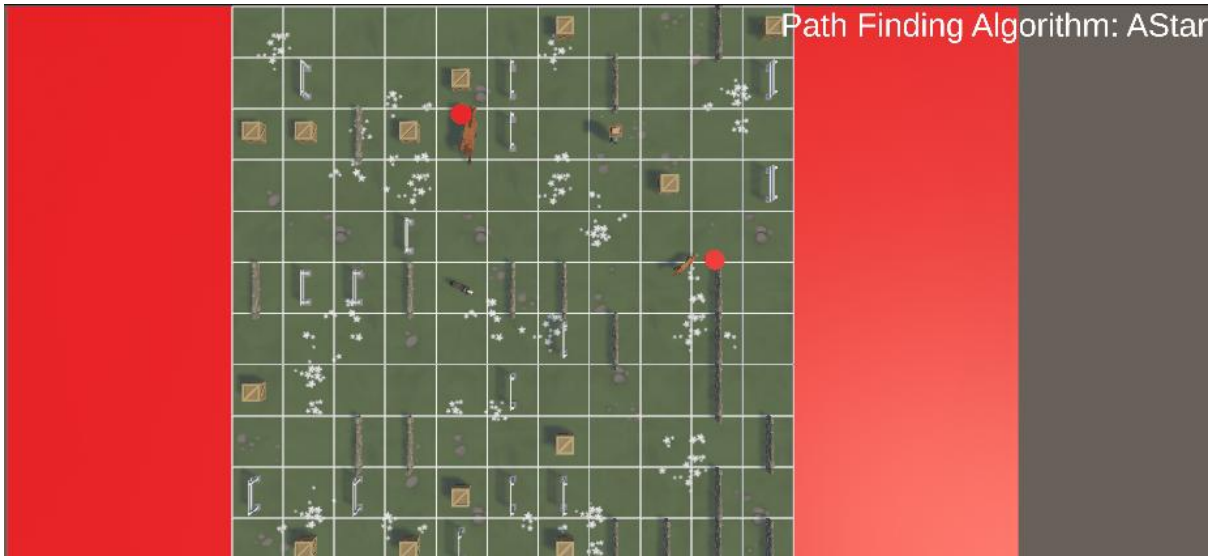
    while (frontier.Count > 0)
    {
        GridCell current = frontier.Dequeue();
        if (current == goal) break;

        foreach (var neighbor in GetNeighbors(current))
        {
            float newCost = costSoFar[current] + Vector2Int.Distance(
                new Vector2Int(current.x, current.z),
                new Vector2Int(neighbor.x, neighbor.z)
            );

            if (!costSoFar.ContainsKey(neighbor) || newCost < costSoFar[neighbor])
            {
                costSoFar[neighbor] = newCost;

                float priority = newCost + Vector2Int.Distance(
                    new Vector2Int(neighbor.x, neighbor.z),
                    new Vector2Int(goal.x, goal.z)
                );

                frontier.Enqueue(neighbor, priority);
                cameFrom[neighbor] = current;
            }
```



☐ **Expert 2: Comparing the algorithms**

Medium2 이슈와 더불어 시간 부족으로 인하여, 모든 셀 탐색 결과를 시각화하는 기능을 구현하지는 못하였습니다. 탐색된 셀과 최종 경로 셀을 분리하여 노란색/흰색으로 나누는 시도는 있었으나, 결국 색을 입히지 못하는 문제를 해결하지 못하여 구현에 실패하였습니다.

■ Conclude with some comments on your work

☐ Key challenges you have successfully tackled

경로 탐색 알고리즘 3종(BFS, Greedy, A*)을 모두 구현하고, Priority Queue를 효과적으로 활용하여 A*의 f(n) 계산과정까지 구현할 수 있었습니다. 베지어 곡선을 따라 움직이는 기능도 구현하며 다양한 시각적 요소와의 통합을 다룰 수 있었고, 플레이어와 동물 간 상호작용을 자연스럽게 연결하는 데 성공하여 주요 도전과제는 전부 달성했다고 생각합니다.

☐ **Limitations you hope to address in the future**

GridCell 색상 변경 문제는 가장 큰 미해결 과제 중 하나로 남았습니다. 렌더러가 머티리얼을 인스턴싱하여 색상 반영이 되지 않는 문제나 셰이더, canvas 레이어 이슈 등 다양한 가능성을 조사하였지만 해결하지 못했고, 기능 구현 여부를 다른 방식으로도 재검토할 필요가 있었습니다. 셀 시각화 기능이 누락되었기에, 향후 비교 및 분석에 도움이 되도록 방문 셀 기록 시각화를 완성하고 싶은 의지가 있습니다.

■ **Add a link to a video that runs the program (e.g. YouTube)**

https://youtu.be/2vH1Il9kB5o